🚦 Performance | Use-As-Dictionary

Project	Use-As-Dictionary
Brands	Cross brand
Points of Contact	
Performance	Web Performance Team (@Giorgio Pellegrino @Christian Silvestri @Daniele Merola @Domenico Cavaglieri @Lorenzo Bartomioli @emanuele.galeotti)
References	Supercharge compression efficiency with shared dictionaries | Blog | Chrome for Developers

Introduction
1. Server-Side Configuration
  Use-As-Dictionary
  Content-Encoding
  Vary
  Dictionary caching
2. Build-Time Dictionary Generation
  Generate Dictionary .dat
  Resource DCB Compression
3. Client-Side Dictionary Discovery

Introduction

The objective of this document is to define an implementation approach within the current Luxottica architecture for serving resources using **dcb** content encoding.

> ✅ The objective of this activity is to compress resources using a dictionary-based compression system, achieving compression rates of up to 90%.

The implementation requires changes across multiple layers of the architecture, including server-side configuration, the application build process, and client-side resource discovery.

1. Server-Side Configuration

The first step is to configure the response headers for the resources that will be eligible for dictionary-based compression:

Use-As-Dictionary

The dictionary resource must return a `Use-As-Dictionary` response header defining which resources can use it as a compression dictionary.

For example:

```
Use-As-Dictionary: match="/dist/styles.*.css"
```

INFRA-2447: Performance RB | Add Use-As-Dictionary Response Header Pending Clarification

The `match` directive supports URL patterns and wildcards, allowing the same dictionary to be used for multiple versioned resources.

Content-Encoding

The CSS resource compressed using the shared Brotli dictionary must be served with:

```
Content-Encoding: dcb
```

⚠ **Only if** the appropriate representation based on the client's `Accept-Encoding` and `Available-Dictionary` request headers. For example, a client supporting dictionary compression may send:

```
Accept-Encoding: gzip, br, zstd, dcb, dcz
Available-Dictionary: :<base64-sha256-hash>:
```

Vary

Because the response varies depending on whether the client supports `dcb` and which dictionary is available, the compressed resource should also include:

```
Vary: Accept-Encoding, Available-Dictionary
```

Dictionary caching

The dictionary resource should be cached by the browser for at least two weeks (average time of luxottica releases).

The recommended response header is therefore:

```
Cache-Control: public, max-age=1209600, immutable
```

2. Build-Time Dictionary Generation

During the application build process, a dictionary file should be generated for the CSS resources and the resources will be compressed.

For example:

```
dictionary_style_v1.dat
```

```
styles.9749.css.dcb
```

> The dictionary URL should ideally also be versioned so that a new dictionary can be introduced without invalidating or conflicting with a previously cached version.

Generate Dictionary .dat

A custom Webpack plugin or a build-time script should be used for the DCB-specific part of the pipeline.

A simplified Webpack integration could look like:

```
const { Compilation, sources } = require("webpack");
const fs = require("fs");
const path = require("path");
const crypto = require("crypto");
const { execFileSync } = require("child_process");

class CompressionDictionaryPlugin {
  apply(compiler) {
    compiler.hooks.thisCompilation.tap(
      "CompressionDictionaryPlugin",
      (compilation) => {
        compilation.hooks.processAssets.tap(
          {
            name: "CompressionDictionaryPlugin",
            stage: Compilation.PROCESS_ASSETS_STAGE_OPTIMIZE_TRANSFER,
          },
          (assets) => {
            const cssAsset = assets["dist/styles.9749.css"];

            if (!cssAsset) {
              return;
            }

            const css = Buffer.from(cssAsset.source());

            const dictionaryPath = path.resolve(
              compiler.options.output.path,
              "dictionary_style_v1.dat"
            );

            // The actual dictionary-generation strategy
            // should be implemented here.
            fs.writeFileSync(dictionaryPath, css);

            const dictionary = fs.readFileSync(dictionaryPath);
            const dictionaryHash = crypto
              .createHash("sha256")
              .update(dictionary)
              .digest();

            // The DCB header consists of:
            // 0xff 0x44 0x43 0x42 + SHA-256(dictionary)
            const dcbHeader = Buffer.concat([
              Buffer.from([0xff, 0x44, 0x43, 0x42]),
              dictionaryHash,
            ]);

```

```
            const compressed = execFileSync("brotli", [
              "--stdout",
              "-D",
              dictionaryPath,
            ], {
              input: css,
            });

            const dcb = Buffer.concat([
              dcbHeader,
              compressed,
            ]);

            compilation.emitAsset(
              "dist/styles.9749.css.dcb",
              new sources.RawSource(dcb)
            );
          }
        );
      }
    );
  }
}
```

Resource DCB Compression

Once the dictionary has been generated, the CSS resource must be compressed using that dictionary.

The resulting asset would be:

```
styles.9749.css.dcb
```

The official **zlib** webpack plugin can be used to perform the underlying dictionary-based compression. MDN provides the following reference implementation:

```
echo -en '\xffDCB' > styles.9749.css.dcb
openssl dgst -sha256 -binary dictionary_style_v1.dat >>
styles.9749.css.dcb
brotli --stdout -D dictionary_style_v1.dat styles.9749.css >>
styles.9749.css.dcb
```

This approach explicitly generates the DCB header and then appends the Brotli stream compressed with the external dictionary.

3. Client-Side Dictionary Discovery

The HTML document should instruct the browser to download and cache the dictionary using the standardized compression-dictionary link relation.

The syntax is:

```
<link rel="compression-dictionary" href="/dictionary_style_v1.dat">
```

The browser can download this dictionary during an appropriate idle period and subsequently use it when requesting matching resources.

The CSS compressed resource can be referenced normally:

```
<link rel="stylesheet" href="/dist/styles.9749.css.dcb">
```

⚠ **Only if** the appropriate representation based on the client's `Accept-Encoding` and `Available-Dictionary` request headers.